

Motores de elementos finitos internos para spinpy: comparación de NGSolve, FEniCSx, scikit-fem y SfePy con el resolvedor de la aplicación

Verificación de las métricas morfométricas y de los algoritmos de optimización
de AppFinal_v2

Indice

Resumen	3
1. Objetivo	4
2. Diseño de la comparación	4
2.1 Un mismo problema discreto para todos	4
2.2 Casos	4
2.3 Medida	5
3. Formulación matemática	5
3.1 Elasticidad lineal	5
3.2 Elementos	6
3.3 Sistema reducido y resolutores	6
3.4 Postproceso común	7
3.5 Análisis no lineal (lagrangiano total)	7
3.6 Soluciones cerradas del bloque uniaxial	8
3.7 Métricas de comparación	9
4. Resultados	9
4.1 Soluciones exactas	9
4.2 Cavidad esférica frente a FEBio	10
4.3 Espinodoide con malla de ladrillos	10
4.4 Espinodoide con malla suave (TET10)	11
4.5 Espinodoide no lineal	13
4.6 Fiabilidad: lo que falló y por qué	13
4.7 Distribución e independencia	14
5. Discusión	15
6. Limitaciones	16
7. Reproducir	16
8. Referencias	16
9. Lista de chequeo de la guía de reporte	17

Fecha: 2026-09-30. spinpy V2.0.0. Código: `spinpy/motores/`, `spinpy/fem.py`. Banco de pruebas: `comparativa_motores/` (`correr.py`, `casos.py`, `comun.py`, `figuras.py`). Datos: `comparativa_motores/resultados/*.jsonl`. Tablas completas: `comparativa_motores/tablas.md`. Pruebas repetibles: bloque 29 de la suite (`tests/test_29_motores.py`).

Resumen

Objetivo. Eliminar la dependencia de spinpy respecto de FEBio, un ejecutable externo, sin perder la malla suave de tetraedros cuadráticos (TET10) ni los análisis no lineales, y con una fiabilidad demostrada y no supuesta.

Métodos. Se integraron en spinpy cuatro bibliotecas de elementos finitos que corren dentro del proceso de la aplicación (NGSolve 6.2, FEniCSx 0.11, scikit-fem 12.0 y SfePy 2026.3) y se compararon con el resolutor propio de la aplicación sobre problemas discretos idénticos: un bloque macizo con solución exacta (hexaedros y TET10), la compresión uniaxial no lineal de un bloque con solución cerrada para St. Venant-Kirchhoff y para el neo-Hookeano de FEBio, una cavidad esférica con la referencia de FEBio guardada en el repositorio, un espinodoide trabecular de 30 672 a 661 965 grados de libertad (GDL) con hexaedros y de 314 880 a 790 599 GDL con TET10, y el mismo espinodoide en régimen no lineal. Se midieron la exactitud, la concordancia entre motores, el tiempo de pared, la memoria de pico y la viabilidad de distribución.

Resultados. En los problemas lineales, los cinco motores dieron el mismo desplazamiento dentro de la tolerancia de sus resolutores con todos los directos y con los iterativos, salvo una excepción descrita más abajo (diferencias máximas de 10^{-13} a 10^{-10} ; 10^{-9} con la app, que resuelve a 10^{-8}). En el bloque macizo todos reprodujeron $E_{app} = E_s$ a 10^{-11} o mejor, y en la compresión no lineal los cuatro motores externos reprodujeron la fuerza cerrada a $4 \cdot 10^{-12}$ hasta el 20 % de acortamiento. Frente a FEBio, la cavidad con hexaedros coincidió a $7,7 \cdot 10^{-9}$ en E_{app} y a $1,7 \cdot 10^{-9}$ en el p99 de von Mises. Hubo tres excepciones de fiabilidad: la tangente de St. Venant-Kirchhoff de SfePy no es la derivada de su residuo (error de 37 %), lo que hace que Newton converja linealmente y no alcance el 2 % de deformación en el espinodoide; el iterativo de FEniCSx (CG + GAMG) divergió en la malla TET10 del espinodoide a 32^3 ; y el CG + pyamg de scikit-fem se detuvo en un residuo de 10^{-10} con un error de desplazamiento de hasta 0,9 % en las mallas TET10 del espinodoide. En tiempo, con TET10 a 790 599 GDL, NGSolve resolvió en 45 s (Cholesky propio, 3,2 GB) o 59 s (CG + BDDC, 3,9 GB), FEniCSx en 22 s (MUMPS, 3,8 GB), y scikit-fem y SfePy en 905 y 1010 s. Solo NGSolve y scikit-fem publican ruedas para Windows en PyPI; FEniCSx requiere conda y un compilador de C en tiempo de ejecución, y SfePy solo publica ruedas para Linux.

Conclusión. NGSolve es el motor adecuado para lo que la aplicación no resolvía por sí misma (TET10 y no lineal): exacto en todas las pruebas, rápido, instalable sin programas externos en Windows, con licencia LGPL-2.1 y sin necesidad de MKL, porque su Cholesky propio fue más rápido y usó menos de la mitad de memoria que PARDISO en todos los casos medidos. El resolutor propio de la app se mantiene para la malla de ladrillos en lineal, donde es competitivo y está validado. FEBio deja de ser necesario; la exportación a `.feb` se conserva como formato de intercambio.

EN PALABRAS SENCILLAS

Con resolutores bien convergidos, los cinco motores resuelven el mismo problema y dan el mismo resultado hasta la décima o duodécima cifra. NGSolve es el que conviene integrar: resuelve la malla suave y el no lineal en segundos o en un minuto, se instala con la aplicación en Windows y no necesita ningún programa externo.

1. Objetivo

Hasta la versión 1.0.2, spinpy resolvía sus ensayos de compresión con dos programas: su propio resolutor sobre hexaedros de un vóxel (`resistencia.ensayo_compresion`) y FEBio 4.5, un ejecutable externo que se lanzaba como proceso aparte para la malla suave de tetraedros cuadráticos (TET10) y para los análisis no lineales. Esa segunda vía hacía a la aplicación dependiente de un programa cuya licencia no permite redistribuirlo. Este informe evalúa cuatro bibliotecas de elementos finitos que pueden correr dentro del proceso de spinpy, sin programas externos, y las compara con el resolutor actual de la aplicación en cinco ejes: exactitud frente a soluciones cerradas, concordancia entre motores sobre la misma malla, tiempo de cálculo, memoria de pico y viabilidad de distribución (licencia, disponibilidad en Windows, dependencias en tiempo de ejecución).

Los cinco motores comparados son:

Motor	Versión	Licencia	Lenguaje del núcleo	Resolutores usados
App (spinpy)	1.0.2 (sin cambios en 2.0.0)	MIT	Python (numpy, scipy, pyamg)	LU de SuperLU por debajo de 6000 GDL; CG + multigrid algebraico (pyamg) con modos rígidos por encima
NGSolve	6.2.2607	LGPL-2.1	C++ con interfaz Python	PARDISO (MKL); CG + BDDC (P2); CG + pyamg sobre su matriz (hex8)
FEniCSx (DOLFINx)	0.11.0	LGPL-3.0	C++ con UFL y compilación JIT	Cholesky de MUMPS; CG + GAMG (PETSc) con modos rígidos
scikit-fem	12.0.2	BSD-3	Python (numpy, scipy)	SuperLU; CG + pyamg con modos rígidos
SfePy	2026.3	BSD-3	Python con extensiones C/Cython	SuperLU (<code>ls.scipy_direct</code>); CG (<code>ls.scipy_iterative</code>) + pyamg con modos rígidos

2. Diseño de la comparación

2.1 Un mismo problema discreto para todos

La comparación aísla el motor. Cada caso es un problema discreto completo que construye spinpy (malla, condiciones de contorno, material y carga) y que se entrega idéntico a los cinco motores (`spinpy.motores.problema_de_malla`). Cada motor devuelve el desplazamiento nodal en el orden de nodos de spinpy; el emparejamiento se hace por coordenadas y se aborta si algún nodo no tiene pareja a 10^{-9} del tamaño de la malla. El postproceso (tensión por elemento, módulo aparente, capa superficial, criterio de Pistoia) es único y se aplica al desplazamiento de cada motor, de modo que cualquier diferencia entre motores procede de su montaje, su cuadratura, sus condiciones de contorno o su resolutor, nunca de la malla ni de la definición de las magnitudes.

Los motores que construyen su propio espacio P2 sobre las esquinas del tetraedro (todos salvo la app, que no resuelve TET10) necesitan aristas rectas. En las mallas TET10 de spinpy, el ajuste de los nodos a los planos del cubo (`fem.preparar_tet10`) mueve esquinas situadas a menos de 10^{-3} h de un plano sin mover el nodo intermedio de sus aristas; se midió un desplazamiento de hasta $8,4 \cdot 10^{-4}$ h respecto del punto medio. Antes de resolver, cada nodo intermedio se recoloca en el punto medio de su arista (ecuación 1), y el desplazamiento máximo aplicado se registra con cada problema.

$$\%eq 1 \\ \mathbf{x}_{ij} = \frac{1}{2} \left(\mathbf{x}_i + \mathbf{x}_j \right), \quad (i, j) \in \{(0, 1), (1, 2), (0, 2), (0, 3), (1, 3), (2, 3)\}$$

2.2 Casos

- 1 **Bloque macizo** de $8 \times 8 \times 12$ vóxeles ($h = 0,05$ mm), con hexaedros y con TET10 (tetgen sobre el mismo cubo). El ensayo lineal tiene solución exacta del problema discreto: deformación uniforme y $E_{app} = E_s$.

- 2 **Bloque en compresión uniaxial no lineal** ($4 \times 4 \times 6$ vóxeles) con plato rígido sin fricción hasta el 20 % de acortamiento, con St. Venant-Kirchhoff y con el neo-Hookeano compresible de FEBio. La deformación es homogénea y la fuerza tiene expresión cerrada (sección 3.6).
- 3 **Cavidad esférica:** cubo de 1 mm con una cavidad centrada de radio 0,2 mm. Con hexaedros a 32^3 (vóxeles) y con TET10 sobre la esfera analítica (la malla de referencia de `comparativa_febio_tet/cavidad.py`, la misma función y los mismos argumentos), para contrastar con el resultado de FEBio guardado en el repositorio.
- 4 **Espinodoide trabecular** ($\rho = 0,30$, número de onda 12π , 700 ondas, conos de 30° , 30° y 90° , semilla 1, lado de 5 mm) con hexaedros a 24^3 , 32^3 , 48^3 , 64^3 y 80^3 . Con la misma semilla el campo aleatorio es el mismo a cualquier resolución; al subir n cambia la discretización, no la estructura.
- 5 **Espinodoide con malla suave TET10** a 32^3 y 48^3 (`fem.malliar`, opciones por omisión: 20 iteraciones de Taubin, decimado 0,5, corrección de volumen).
- 6 **Espinodoide no lineal** a 24^3 con hexaedros, plato rígido al 0,25 %, 0,5 %, 1 % y 2 %, con los dos materiales.

Material: $E_s = 20$ GPa, $\nu = 0,30$. Ensayo de la app: tensión aparente de 1 MPa sobre la sección bruta, apoyo deslizante.

2.3 Medida

Cada resolución corre en un proceso hijo (`comparativa_motores/correr.py`), con 4 hilos permitidos (`OMP_NUM_THREADS = MKL_NUM_THREADS = 4`) en un equipo de 4 núcleos y 15 GB, sin otra carga durante la campaña. El tiempo es de reloj de pared por etapas (malla del motor, compilación JIT, montaje, resolución, extracción) y excluye la importación del motor. En FEniCSx, la compilación JIT de las formas se informa aparte y solo ocurre la primera vez que se usa cada forma (después queda en caché en disco). La memoria es el pico de memoria residente del proceso hijo leído de `VmHWM` (no de `ru_maxrss`, que en Linux se hereda a través de `exec` y daría el del proceso lanzador). Un vigilante detiene el hijo por encima de 13 GB o de una hora.

3. Formulación matemática

3.1 Elasticidad lineal

Con desplazamiento $\mathbf{u}$, deformación infinitesimal y ley de Hooke isótropa (Lamé λ, μ):

$$\begin{aligned} \varepsilon_{ij} &= \frac{1}{2} (\nabla_i u_j + \nabla_j u_i) \\ \sigma_{ij} &= \lambda \text{tr}(\varepsilon) \delta_{ij} + 2\mu \varepsilon_{ij} \end{aligned}$$

En notación de Voigt (orden xx, yy, zz, yz, xz, xy, con distorsiones de ingeniería $\gamma = 2\varepsilon$), $\boldsymbol{\sigma} = \mathbf{D} \boldsymbol{\varepsilon}$ con

$$\mathbf{D} = \frac{1}{1+\nu} \begin{bmatrix} \lambda+2\mu & \lambda & \lambda & 0 & 0 & 0 \\ \lambda & \lambda+2\mu & \lambda & 0 & 0 & 0 \\ \lambda & \lambda & \lambda+2\mu & 0 & 0 & 0 \\ 0 & 0 & 0 & \mu & 0 & 0 \\ 0 & 0 & 0 & 0 & \mu & 0 \\ 0 & 0 & 0 & 0 & 0 & \mu \end{bmatrix}$$

La forma débil del ensayo de compresión es: hallar $\mathbf{u}$ que cumpla las condiciones esenciales tal que, para todo $\mathbf{v}$ admisible,

$$\int_{\Omega} \boldsymbol{\sigma}(\mathbf{u}) : \boldsymbol{\varepsilon}(\mathbf{v}) \, d\Omega = \int_{\Gamma} \boldsymbol{\sigma}(\mathbf{u}) \cdot \mathbf{n} \cdot \mathbf{v} \, d\Gamma - p \int_{\Omega} \mathbf{e}_z \cdot \mathbf{v} \, d\Omega$$

donde σ_0 es la tensión aparente sobre la sección bruta A_b y A_t es el área ósea de las caras del techo, de modo que la fuerza total es $\sigma_0 A_b$, la convención de la app.

Condiciones de contorno. Apoyo deslizante: $u_z = 0$ en todos los nodos de la base (intermedios incluidos con TET10); $u_x = u_y = 0$ en el nodo de la base que minimiza $x + y$ y $u_y = 0$ en el que maximiza $x - y$, lo mínimo para eliminar los movimientos de sólido rígido. Con TET10 las dos anclas se eligen entre las esquinas. Apoyo empotrado: los tres componentes en toda la base. Plato rígido sin fricción: además, $u_z = -\epsilon_p H$ en todos los nodos del techo.

3.2 Elementos

Hexaedro trilineal (hex8). Sobre el cubo de referencia $\xi, \eta, \zeta \in [-1, 1]$, con los vértices en el orden C3D8 ($\xi_a, \eta_a, \zeta_a \in \{-1, 1\}$):

$$N_a(\xi, \eta, \zeta) = \frac{1}{8}(1 + \xi_a \xi)(1 + \eta_a \eta)(1 + \zeta_a \zeta), \quad a=1, \dots, 8$$

Integración de Gauss $2 \times 2 \times 2$ (puntos $\pm 1/\sqrt{3}$, pesos 1). En un vóxel el jacobiano es constante, $J = \text{diag}(h_x, h_y, h_z)/2$.

Tetraedro cuadrático (TET10). Con las coordenadas baricéntricas L_i ($i = 0, \dots, 3$) y el orden C3D10 (aristas 01, 12, 02, 03, 13, 23):

$$N_i = L_i(2L_i - 1), \quad N_{ij} = 4L_i L_j$$

Con aristas rectas el mapeo es afín, la deformación es lineal en el elemento y la matriz de rigidez (integrando de grado 2) se integra exactamente con cualquier regla de grado ≥ 2 . Cada motor usa la suya: scikit-fem, FEniCSx y SfePy, grado 2; NGSolve, grado 4. Todas son exactas para este integrando, así que el sistema lineal es el mismo. Con hexaedros de un vóxel el mapeo también es afín y el integrando es de grado 2 en cada dirección; todos los motores usan Gauss con dos puntos por dirección, exacto hasta grado 3.

Matriz de rigidez y carga. Con $\mathbf{B}$ la matriz de derivadas de forma en Voigt,

$$\mathbf{K} = \sum_e \int_{\Omega_e} \mathbf{B}^T \mathbf{D} \mathbf{B} \, d\Omega, \quad \mathbf{f}_a = \int_{\Gamma_a} \mathbf{N}_a \mathbf{t} \, d\Gamma$$

En una cara cuadrada del techo cada uno de sus cuatro nodos recibe $p A/4$ (el reparto por área tributaria de la app); en una cara tri6 las esquinas reciben cero y cada nodo intermedio $p A/3$.

3.3 Sistema reducido y resolutores

Con los grados de libertad particionados en libres (f) y prescritos (d), el sistema que se resuelve es

$$\mathbf{K}_{ff} \mathbf{u}_f = \mathbf{f}_f - \mathbf{K}_{fd} \mathbf{u}_d$$

Los resolutores directos factorizan $\mathbf{K}_{ff}$ (LU de SuperLU con ordenación COLAMD; Cholesky de MUMPS; LDL^T de PARDISO con disección anidada). Los iterativos usan gradiente conjugado preconditionado por multigrid: agregación suavizada (pyamg, GAMG de PETSc) con los seis modos de sólido rígido como espacio casi nulo, o BDDC (NGSolve, con TET10). Los modos rígidos son las tres traslaciones y los tres giros infinitesimales:

$$\mathbf{r}_{1..3} = \mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z; \quad \mathbf{r}_4 = (-y, x, 0), \quad \mathbf{r}_5 = (0, -z, y), \quad \mathbf{r}_6 = (z, 0, -x)$$

El criterio de parada es el residuo relativo no preconditionado (ecuación 10), 10^{-10} en los motores externos y 10^{-8} en la app (su valor de siempre). En PETSc se fijó explícitamente la norma no preconditionada, que no es la que usa por omisión.

$$\%eq 10$$

$$r = \frac{|\mathbf{f}| - \mathbf{K}_{ff} \mathbf{u}_f|_{\{2\}}}{|\mathbf{f}|_{\{2\}}}$$

3.4 Postproceso común

La deformación de cada elemento se evalúa en su centro (hex8, la convención de la app, igual a la media de volumen en un vóxel) o en su centroide (TET10, igual a la media de volumen porque la deformación es lineal); la tensión es $\sigma_e = \mathbf{D} \varepsilon_e$. De ella salen la tensión de von Mises y la deformación efectiva de Pistoia:

$$\%eq 11$$

$$\sigma_{vM} = \sqrt{\frac{1}{2} \left[(\sigma_{xx} - \sigma_{yy})^2 + (\sigma_{yy} - \sigma_{zz})^2 + (\sigma_{zz} - \sigma_{xx})^2 \right] + 3 \left[\tau_{yz}^2 + \tau_{xz}^2 + \tau_{xy}^2 \right]}$$

$$\%eq 12$$

$$U_e = \frac{1}{2} \mathbf{u}_e \cdot \mathbf{e}, \quad \text{quadr} \quad \varepsilon_{eff,e} = \sqrt{\frac{2}{U_e} \{E_s\}}$$

El módulo aparente usa el desplazamiento vertical medio del techo: la media simple de sus nodos (la definición de la app con vóxeles) o la media ponderada por área de sus caras cargadas (la que se usa con TET10):

$$\%eq 13$$

$$E_{app} = \frac{\sigma_0}{|\bar{u}_z|/H}, \quad \text{quadr} \quad \bar{u}_z^{nodal} = \frac{1}{n_t} \sum_{k \in \text{techo}} u_{z,k}, \quad \text{quadr} \quad \bar{u}_z^{area} = \frac{\sum_c A_c}{|\bar{u}_z|}, \quad \text{quadr} \quad \sum_c A_c$$

El equilibrio se comprueba, con independencia del motor, por la integral de volumen de la tensión (exacta en el problema discreto lineal, con el desplazamiento virtual $v = z e_z$):

$$\%eq 14$$

$$F = -\frac{1}{H} \sum_e \sigma_{zz,e} V_e, \quad \text{quadr} \quad dF = \frac{|F - \sigma_0 A_b|}{|\sigma_0 A_b|}$$

El criterio de Pistoia et al. (2002) busca el factor k que lleva el percentil 98 (en volumen) de ε_{eff} a la deformación crítica, y el p99 de von Mises se toma en la capa superficial (elementos con una cara hacia el poro, sin contar las caras del cubo), ambos ponderados por volumen con TET10:

$$\%eq 15$$

$$k = \frac{\varepsilon_{crit}}{P_{98}(\varepsilon_{eff})}, \quad \text{quadr} \quad \sigma_{fallo} = k \sigma_0, \quad \text{quadr} \quad \varepsilon_{crit} = 0.007$$

3.5 Análisis no lineal (lagrangiano total)

Con el gradiente de deformación $\mathbf{F}$, el tensor de Cauchy-Green derecho $\mathbf{C}$ y el de Green-Lagrange $\mathbf{E}$:

$$\%eq 16$$

$$\mathbf{F} = \mathbf{I} + \nabla_0 \mathbf{u}, \quad \text{quadr} \quad \mathbf{C} = \mathbf{F}^T \mathbf{F}, \quad \text{quadr} \quad \mathbf{E} = \frac{1}{2} (\mathbf{C} - \mathbf{I}), \quad \text{quadr} \quad J = \det \mathbf{F}$$

Los dos materiales hiperelásticos comparados son St. Venant-Kirchhoff (el isotropic elastic de FEBio) y el neo-Hookeano compresible de FEBio:

$$\%eq 17$$

$$W_{SVK} = \frac{\lambda}{2} (\text{tr} \mathbf{E})^2 + \mu \text{tr} \mathbf{E}, \quad \text{quadr} \quad W_{NH} = \frac{\mu}{2} (\text{tr} \mathbf{C} - 3) - \mu \ln J + \frac{\lambda}{2} (\ln J)^2$$

Sus segundos tensores de Piola-Kirchhoff $\mathbf{S} = \partial \mathbf{W} / \partial \mathbf{E}$ son

$$\begin{aligned} \text{\%eq 18} \\ \mathbf{S}_{\text{SVK}} = \lambda \text{tr}(\mathbf{E}) \mathbf{I} + 2\mu \mathbf{E}, \quad \mathbf{S}_{\text{NH}} = \mu (\mathbf{I} - \mathbf{C}^{-1}) + \lambda \ln J \mathbf{I}, \quad \mathbf{C}^{-1} \end{aligned}$$

El residuo (fuerzas internas menos externas) con el primer tensor de Piola-Kirchhoff $\mathbf{P} = \mathbf{F} \mathbf{S}$ es

$$\text{\%eq 19} \\ \mathbf{R}(\mathbf{u}) \cdot \mathbf{v} = \int_{\Omega_0} \mathbf{F} \mathbf{S} : \nabla_0 \mathbf{v} \, d\Omega_0 - \int_{\Gamma} \mathbf{t} \cdot \mathbf{v} \, d\Gamma$$

y su linealización consistente en la dirección $\delta \mathbf{u}$, con $\delta \mathbf{E} = \text{sym}(\mathbf{F}^T \nabla_0 \delta \mathbf{u})$, es

$$\text{\%eq 20} \\ D\mathbf{R}[\delta \mathbf{u}] \cdot \mathbf{v} = \int_{\Omega_0} \left(\nabla_0 \delta \mathbf{u} : \mathbf{S} + \mathbf{F} \cdot \nabla_0 \delta \mathbf{u} \right) : \nabla_0 \mathbf{v} \, d\Omega_0$$

$$\text{\%eq 21} \\ \begin{aligned} \delta \mathbf{S}_{\text{SVK}} &= \lambda \text{tr}(\delta \mathbf{E}) \mathbf{I} + 2\mu \delta \mathbf{E}, \quad \delta \mathbf{S}_{\text{NH}} = \lambda (\mathbf{C}^{-1} : \delta \mathbf{E}) \mathbf{I} + (\mu - \lambda \ln J) \mathbf{C}^{-1} \delta \mathbf{E}, \quad \mathbf{C}^{-1} \end{aligned}$$

El primer término de la ecuación 20 es la rigidez geométrica. scikit-fem recibe las ecuaciones 18 a 21 escritas a mano; NGSolve y FEniCSx derivan la energía W automáticamente (Variation y derivative de UFL); SfePy usa su término `dw_t_l_he_sv_k`. Cada paso de carga se resuelve por Newton-Raphson,

$$\text{\%eq 22} \\ \mathbf{K}_T(\mathbf{u}^k) \Delta \mathbf{u} = -\mathbf{R}(\mathbf{u}^k), \quad \mathbf{u}^{k+1} = \mathbf{u}^k + \Delta \mathbf{u}, \quad \frac{\|\mathbf{R}(\mathbf{u}^k)\|}{\|\mathbf{f}_{\text{int, techo}}\|} \leq 10^{-10}$$

Predictor consistente del plato. Con control por desplazamiento, imponer el incremento $\Delta \mathbf{u}_d$ solo en los nodos del techo deja una primera iteración muy distorsionada. En el bloque al 20 % con St. Venant-Kirchhoff, cuya energía no es convexa, el Newton que se escribió así para NGSolve convergió en 22 iteraciones a otra rama de equilibrio (fuerza de -133,4 N en lugar de 115,2 N). La primera iteración de cada paso se hace, por eso, con el incremento del plato dentro del sistema linealizado, el mismo esquema que aplica SNES de PETSc con las condiciones de contorno por elevación:

$$\text{\%eq 23} \\ \mathbf{K}_{T,ff} \Delta \mathbf{u}_f = -\mathbf{R}_f - \mathbf{K}_{T,fd} \Delta \mathbf{u}_d$$

Fuerza de reacción. Es la suma de las fuerzas internas en z sobre los nodos del techo, positiva en compresión:

$$\text{\%eq 24} \\ F_{\text{reac}} = -\sum_{k \in \text{techo}} f_{\text{int}, z, k}$$

Carga muerta frente a presión seguidora. Con control por fuerza, los motores internos aplican una tracción muerta (fija en la configuración de referencia); la presión de FEBio sigue a la cara deformada. La diferencia es del orden de la rotación de las caras del techo y se declara en el informe de la aplicación.

3.6 Soluciones cerradas del bloque uniaxial

Con plato sin fricción, base deslizante y laterales libres, la deformación es homogénea, $\mathbf{F} = \text{diag}(a, a, \lambda_z)$, y $S_{xx} = S_{yy} = 0$ fija el estiramiento lateral a . Para St. Venant-Kirchhoff la solución es explícita:

```
%eq 25
E_{xx}=-\nu\,E_{zz},\quad E_{zz}=\frac{\lambda_{zz}^2-1}{2},\quad P_{zz}=\lambda_{zz}\,E,\quad \frac{1}{\lambda_{zz}^2-1},\quad F=-P_{zz}A_0
```

Para el neo-Hookeano de FEBio, a es la raíz de la ecuación 26 (resuelta por el método de Brent a 10^{-15}) y la tensión nominal es la ecuación 27:

```
%eq 26
\mu(a^2-1)+\lambda\ln(a^2\lambda_{zz})=0
```

```
%eq 27
P_{zz}=\lambda_{zz}\left[\mu\left(1-\lambda_{zz}^{-2}\right)+\lambda\ln J,\right],\quad J=a^2\lambda_{zz}
```

3.7 Métricas de comparación

Entre dos soluciones del mismo problema discreto (un motor frente a la referencia del caso, que es una solución por resolutor directo; la de NGSolve si existe, si no la de FEniCSx, SfePy o scikit-fem):

```
%eq 28
\Delta u=\frac{\max_k|\mathbf{u}_k-\mathbf{u}_k^{\text{ref}}|}{\max_k|\mathbf{u}_k^{\text{ref}}|},\quad \Delta\sigma_{\text{VM}}=\frac{\max_e|\sigma_{\text{VM},e}-\sigma_{\text{VM},e}^{\text{ref}}|}{\max_e|\sigma_{\text{VM},e}^{\text{ref}}|},\quad \Delta E=\frac{E_{\text{app}}-E_{\text{app}}^{\text{ref}}}{E_{\text{app}}^{\text{ref}}}
```

Para comprobar la tangente de un motor se comparó su matriz tangente con la derivada numérica centrada de su propio residuo, en un estado aleatorio $\mathbf{x}$ y una dirección aleatoria $\mathbf{d}$:

```
%eq 29
e_T=\frac{\left|\mathbf{K}_T(\mathbf{x})-\frac{\mathbf{R}(\mathbf{x}+h\mathbf{d})-\mathbf{R}(\mathbf{x}-h\mathbf{d})}{2h}\right|}{\left|\frac{\mathbf{R}(\mathbf{x}+h\mathbf{d})-\mathbf{R}(\mathbf{x}-h\mathbf{d})}{2h}\right|},\quad h=10^{-7}H
```

Una tangente consistente da e_T del orden del error de truncamiento de la diferencia centrada ($\blacksquare 10^{-8}$); un valor de orden uno indica una tangente que no es la derivada del residuo, y con ella Newton pierde la convergencia cuadrática.

4. Resultados

Las tablas completas, una fila por caso, motor y resolutor, con tiempo por etapas, memoria, residuo, iteraciones y magnitudes, están en `tablas.md`. Aquí se resumen.

4.1 Soluciones exactas

Bloque macizo, lineal. Con hexaedros (3 159 GDL) y con TET10 (13 446 GDL), los cinco motores y todos sus resolutores dieron $E_{\text{app}} = 20\,000$ MPa, con una diferencia máxima de desplazamientos frente a la referencia de $1,6\cdot 10^{-10}$ (CG + GAMG de FEniCSx, TET10) y de 10^{-13} a 10^{-15} con los directos. La fuerza de equilibrio (ecuación 14) coincidió con la aplicada a $1,3\cdot 10^{-12}$ o mejor en todos los casos. La app resolvió el bloque de hexaedros con su LU directo a $3,4\cdot 10^{-13}$ de la referencia.

Compresión uniaxial no lineal. En el bloque de $4 \times 4 \times 6$ vóxeles con plato hasta el 20 %, las fuerzas de reacción fueron 37,05; 68,40; 94,35 y 115,20 N con St. Venant-Kirchhoff y 41,6387; 86,9777; 136,776 y 192,004 N con el neo-Hookeano, iguales a las cerradas (ecuaciones 25 a 27) con un error máximo de $4,1\cdot 10^{-12}$ (tolerancia de Newton) en scikit-fem, NGSolve y FEniCSx, y de $4,4\cdot 10^{-16}$ en SfePy. scikit-fem, NGSolve y FEniCSx necesitaron tres iteraciones de Newton por paso; SfePy, entre 22 y 27 (sección 4.6).

4.2 Cavidad esférica frente a FEBio

Con hexaedros a 32^3 (105 558 GDL), los cinco motores dieron $E_{app} = 18\,645,20434$ MPa (definición por área) y un p99 de von Mises en la pared de $1,8775142\,\sigma_0$, con diferencias entre motores por debajo de 10^{-12} . FEBio 4.5, sobre la misma malla, había dado $18\,645,20420$ MPa y $1,8775142\,\sigma_0$ (comparativa_febio_tet/resultados/cavidad.jsonl): diferencias de $7,7\cdot 10^{-9}$ y $1,7\cdot 10^{-9}$, compatibles con la extrapolación a carga nula que exigía FEBio.

Con TET10 sobre la esfera analítica (206 142 GDL), los motores coincidieron entre sí a $4\cdot 10^{-10}$ en desplazamientos y dieron $E_{app} = 18\,647,0456$ MPa y un p99 de $2,0965\,\sigma_0$. La referencia de FEBio con la misma función de mallado fue $18\,647,0964$ MPa y $2,0944\,\sigma_0$. La malla no es idéntica (43 690 elementos aquí, 43 230 en la corrida de FEBio: la versión de tetgen es otra), de modo que la diferencia ($-2,7\cdot 10^{-6}$ en E_{app} , $+0,10\%$ en el p99) mezcla malla y motor; su magnitud es la de dos mallas distintas de la misma geometría, no la de un error de implementación.

4.3 Espinodoide con malla de ladrillos

La Tabla 1 resume el tiempo de montaje y resolución (sin importar el motor) y la memoria de pico. Los seis tamaños dan el mismo E_{app} en todos los motores hasta la sexta cifra (326,534; 261,146; 298,888; 280,474 y 264,850 MPa de 24^3 a 80^3), y la diferencia de desplazamientos frente a la referencia directa quedó entre $2\cdot 10^{-13}$ y $4\cdot 10^{-11}$ en los motores externos y entre $4\cdot 10^{-10}$ y $9\cdot 10^{-10}$ en la app (Figura 5).

Tabla 1. Espinodoide hex8: tiempo (s) y memoria de pico (GB) por motor y resolutor.

GD L	App (CG + AMG)	NGSolve Cholesky	NGSolve PARDISO	NGSolve CG + pyamg	FEniCSx MUMPS	FEniCSx CG + GAMG	SfePy CG + pyamg	scikit-fem CG + pyamg
30 672	3,9 / 0,18		1,1 / 0,36	5,1 / 0,33	0,6 / 0,24	3,4 / 0,20	4,8 / 0,25	8,3 / 0,24
62 595	8,0 / 0,31		2,1 / 0,58	10,8 / 0,49	1,6 / 0,34	7,0 / 0,25	11,1 / 0,36	19,3 / 0,45
173 484	28,6 / 0,68	5,3 / 0,67	7,6 / 1,55	42,9 / 1,10	5,2 / 0,79	29,3 / 0,42	43,8 / 0,77	81,4 / 1,27
365 655	60,1 / 1,41	14,4 / 1,54	19,6 / 3,72	70,4 / 2,25	13,2 / 1,72	62,5 / 0,73	69,6 / 1,57	165,9 / 2,98
661 965	128,0 / 2,56	32,7 / 3,35	51,7 / 7,92	141,2 / 4,09	27,7 / 3,53	118,0 / 1,21	149,3 / 2,79	388,5 / 5,66

Con SuperLU (el directo de scikit-fem y SfePy), el tiempo creció de 7 y 5 s a 30 672 GDL a 516 y 923 s, con 7,3 y 7,0 GB, en la cavidad de 105 558 GDL; por encima de $1,5\cdot 10^5$ GDL no se corrió.

Espinodoide hex8: tiempo de montaje y resolución

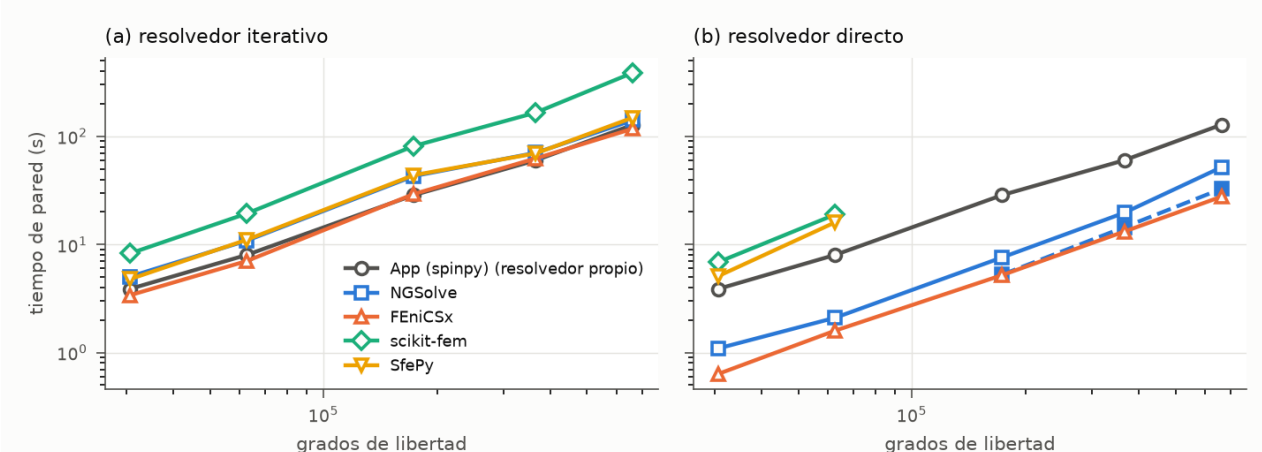


Figura 1. Tiempo de montaje y resolución del espinodoide con hexaedros frente a los grados de libertad, por motor. (a) Resolutores iterativos; (b) directos. La app aparece en los dos paneles con su resolutor propio (LU por debajo de 6 000 GDL, CG + AMG por encima). Línea discontinua: NGSolve con su Cholesky propio.

Espinodoide hex8: memoria de pico del proceso

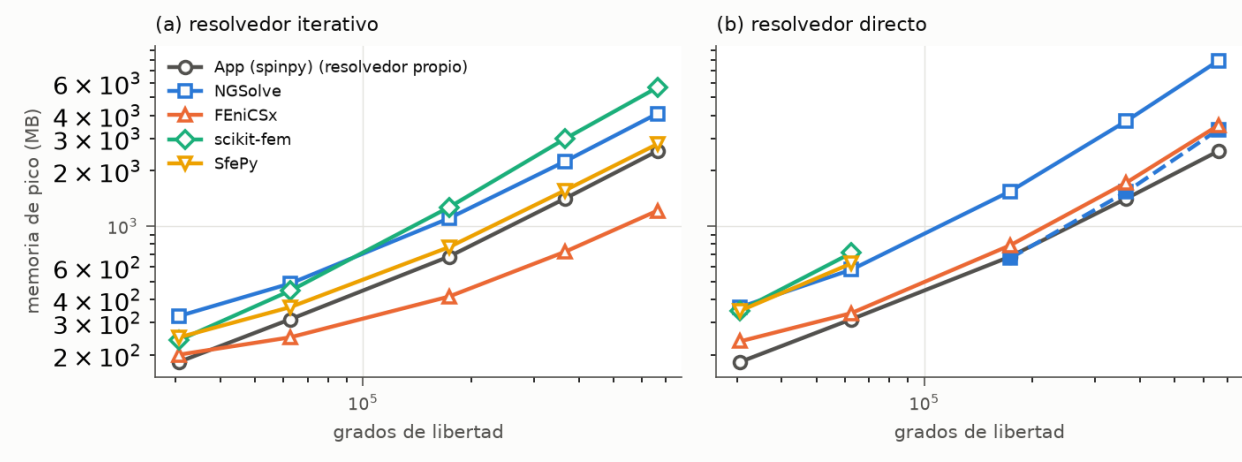


Figura 2. Memoria de pico del proceso de cálculo en los mismos casos.

4.4 Espinodoide con malla suave (TET10)

Tabla 2. Espinodoide TET10: tiempo (s), memoria de pico (GB) y diferencia de desplazamientos frente a la referencia directa.

Motor y resolutor	314 880 GDL (32^3)	790 599 GDL (48^3)
NGSolve, Cholesky propio	13,5 s · 1,04 GB · 0 (ref.)	44,8 s · 3,24 GB · 0 (ref.)
NGSolve, CG + BDDC	18,3 s · 1,30 GB · $5,6 \cdot 10^{-12}$	58,7 s · 3,90 GB · $1,5 \cdot 10^{-12}$
NGSolve, PARDISO	19,5 s · 2,94 GB · 0 (ref.)	69,2 s · 8,31 GB · 0 (ref.)
FEniCSx, MUMPS	7,5 s · 1,41 GB · $4,1 \cdot 10^{-12}$	22,3 s · 3,77 GB · $6,9 \cdot 10^{-12}$
FEniCSx, CG + GAMG	no converge (precondicionador indefinido)	325,0 s · 1,48 GB · $5,7 \cdot 10^{-12}$
SfePy, CG + pyamg	236,2 s · 1,40 GB · $1,8 \cdot 10^{-11}$	1009,7 s · 3,26 GB · $8,2 \cdot 10^{-12}$
scikit-fem, CG + pyamg	258,4 s · 3,02 GB · $8,9 \cdot 10^{-3}$	905,1 s · 7,05 GB · $1,6 \cdot 10^{-3}$

Los motores que convergieron coinciden en E_{app} (156,683 MPa a 32^3 y 219,024 MPa a 48^3), en el p99 de von Mises de la capa superficial (85,767 y 79,029 MPa) y en la tensión de fallo de Pistoia (2,7196 y 3,0729 MPa). scikit-fem con su iterativo se apartó: 157,039 MPa (+0,23 %) y un error de von Mises de hasta 7 % en algún elemento (sección 4.6).

Espinodoide TET10: tiempo de montaje y resolución

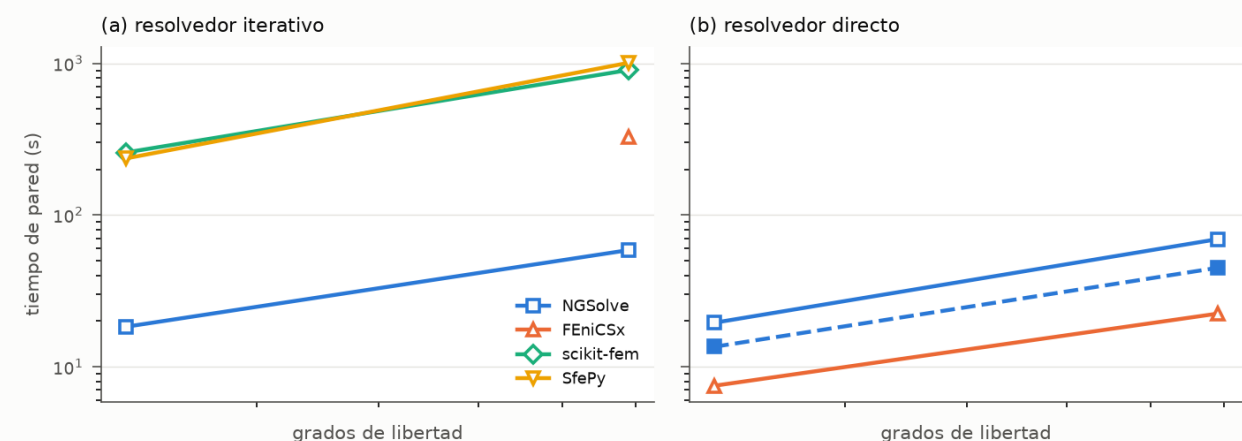


Figura 3. Tiempo del espinodoide con TET10. (a) Iterativos; (b) directos (línea discontinua: NGSolve con su Cholesky propio).

Espinodoide TET10: memoria de pico del proceso

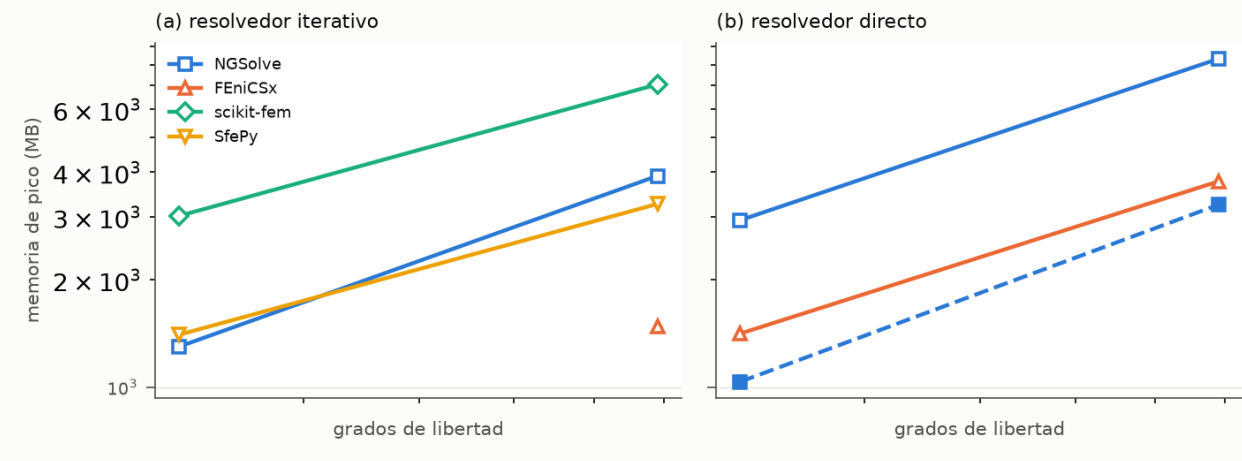


Figura 4. Memoria de pico con TET10.

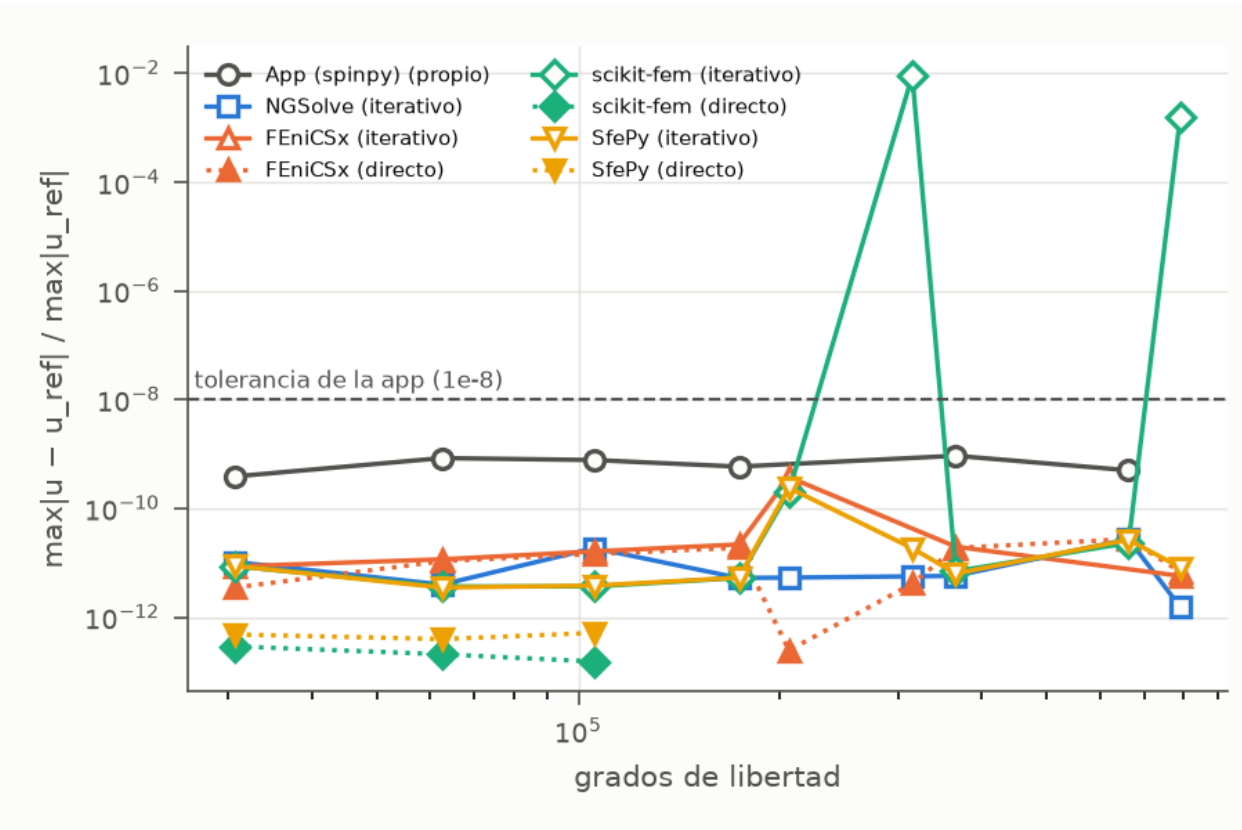


Figura 5. Diferencia máxima de desplazamientos de cada motor y resolutor frente a la referencia directa del caso, en todos los casos lineales del espinodoide y de la cavidad. Línea discontinua: la tolerancia del resolutor de la app. Los dos picos de scikit-fem son sus soluciones iterativas en las mallas TET10.

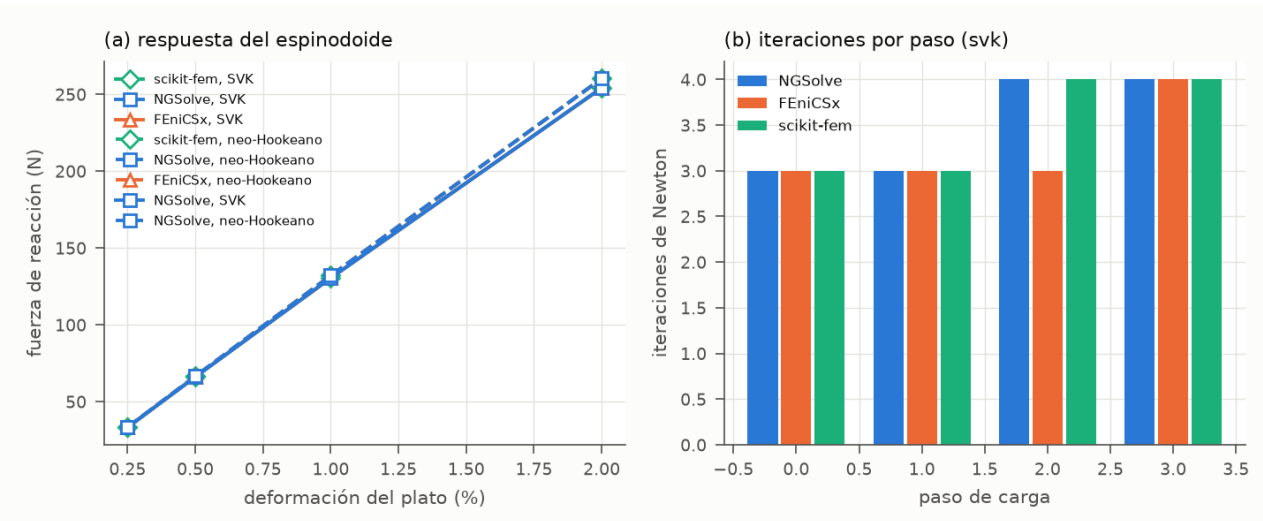
Calidad de las mallas TET10. La Tabla 3 resume la forma de los tetraedros, medida sobre sus cuatro esquinas: la relación radio-arista (radio de la esfera circunscrita dividido por la arista más corta; 0,612 en el tetraedro regular) y el menor ángulo diedro de cada elemento. En ambas resoluciones la mediana de la relación radio-arista fue 1,04 y la del diedro mínimo, de 37° a 38° ; alrededor del 4 % de los elementos superó una relación de 2 y el 1 % tuvo un diedro menor de 5° . Hubo astillas aisladas (diedro mínimo de $0,47^\circ$ a $0,68^\circ$; menos de seis elementos por debajo de 1°). La malla de 32^3 contiene además 577 tetraedros con volumen inferior a 10^{-6} veces la mediana; no son planos (mediana de su diedro mínimo, $34,9^\circ$) sino diminutos, con aristas del orden del 0,4 % de la arista típica. La de 48^3 no tiene ninguno.

Tabla 3. Calidad de las mallas TET10 del espinodoide (esquinas de cada tetraedro).

Medida	32 ³ (54 741 TET10)	48 ³ (143 208 TET10)
Relación radio-arista: mediana; p95; p99; máximo	1,04; 1,91; 3,29; 65,0	1,04; 1,86; 3,31; 91,5
Elementos con relación radio-arista > 2 / > 10	4,1 % / 0,11 %	3,6 % / 0,11 %
Diedro mínimo: menor; p1; p5; mediana	0,68°; 5,0°; 12,5°; 37,1°	0,47°; 4,7°; 13,0°; 38,2°
Elementos con diedro mínimo < 5° / < 1°	1,0 % / 1 elemento	1,1 % / 5 elementos
Elementos con volumen < 10 ⁻⁶ × mediana	577	0

4.5 Espinodoide no lineal

Con el plato al 0,25 %, 0,5 %, 1 % y 2 % de acortamiento (30 672 GDL), scikit-fem, NGSolve y FEniCSx dieron las mismas fuerzas: 33,2523; 66,0776; 130,454 y 254,156 N con St. Venant-Kirchhoff y 33,3520; 66,4736; 132,016 y 260,223 N con el neo-Hookeano, con diferencias entre motores de 10⁻¹⁴ a 10⁻¹⁰ (la mayor, en FEniCSx, es la tolerancia relativa de SNES). Todos necesitaron tres o cuatro iteraciones por paso (Figura 6). Los tiempos fueron 4,2 s (FEniCSx, sin la compilación JIT), 9,1 s (NGSolve, Cholesky propio y energía compilada), 18,4 s (NGSolve con PARDISO y energía sin compilar) y 189 s (scikit-fem, que monta en Python). SfePy agotó las 30 iteraciones al 2 % y no dio resultado.


Figura 6. Figura 6. (a) Fuerza de reacción del espinodoide frente al acortamiento del plato, por motor y material: las curvas se superponen. (b) Iteraciones de Newton por paso con St. Venant-Kirchhoff.

La respuesta se aparta de la proporcional ya a estas deformaciones: extrapolando linealmente el primer paso, la fuerza al 2 % sería $8 \times 33,25 = 266,0$ N con St. Venant-Kirchhoff y $8 \times 33,35 = 266,8$ N con el neo-Hookeano, frente a los 254,2 N (-4,5 %) y 260,2 N (-2,5 %) calculados. La diferencia entre los dos materiales, que coinciden en pequeñas deformaciones, indica que parte del ablandamiento procede de la ley constitutiva y no solo de la geometría; este diseño no permite separar ambas contribuciones con más precisión.

4.6 Fiabilidad: lo que falló y por qué

- 1 **Tangente de SfePy.** La comparación de la matriz tangente de `dw_tl_he_svk` con la derivada numérica de su propio residuo (ecuación 29) dio $e_T = 0,37$, frente a $2 \cdot 10^{-11}$ para `dw_lin_elastic` en el mismo estado (`verificar_tangente_sfepy.py`). El residuo es correcto (la fuerza converge a la cerrada), pero la tangente no es su derivada y Newton converge linealmente, con un factor de contracción del residuo de $\approx 0,45$ por iteración. En el espinodoide eso no basta para converger en 30 iteraciones al 2 %. Además, el neo-Hookeano de SfePy es la variante desacoplada ($\mu/2 (J^{-2/3} I_1 - 3) + K/2 (J - 1)^2$), que no es el de FEBio, y se declaró no disponible para no mezclar modelos.

- 2 **GAMG de FEniCSx con TET10.** Con la receta de los ejemplos de elasticidad de DOLFINx (CG, GAMG con modos rígidos, suavizador de Chebyshev), la malla TET10 del espinodoide a 32^3 dio un preconditionador indefinido (KSP_DIVERGED_INDEFINITE_PC). La malla contiene tetraedros casi degenerados junto a la superficie suavizada (volumen mínimo de $1,5 \cdot 10^{-13} \text{ mm}^3$ frente a una mediana de $5 \cdot 10^{-4}$). En el motor integrado, el modo automático pasa entonces a MUMPS.
- 3 **CG + pyamg de scikit-fem con TET10.** En las mallas TET10 del espinodoide se detuvo en el residuo pedido ($9 \cdot 10^{-11}$) con un error de desplazamiento de $8,9 \cdot 10^{-3}$ a 32^3 y de $1,6 \cdot 10^{-3}$ a 48^3 . El mismo preconditionador dentro del CG de SfePy convergió a 10^{-11} . La cota $\|e\|_{\text{H}^1} \leq \kappa(K) \|r\|_{\text{H}^1}$ admite ese error cuando el sistema está mal condicionado, y el criterio de residuo, por sí solo, no lo detecta. Un experimento adicional sobre la malla de 32^3 lo confirma. La malla contiene 577 tetraedros diminutos, con volumen inferior a 10^{-6} veces la mediana (el menor, $2,9 \cdot 10^{-10}$ veces), además de algunas astillas (Tabla 3). Con tolerancia 10^{-10} el CG terminó en 397 iteraciones con residuo $1,0 \cdot 10^{-10}$ y el mismo error de $8,9 \cdot 10^{-3}$; al exigir 10^{-12} no convergió (residuo final $3,6 \cdot 10^{-4}$ tras el máximo de iteraciones). Endurecer la tolerancia no corrige, por tanto, el error: el sistema está demasiado mal condicionado para ese preconditionador. El error fue menor a 48^3 ($1,6 \cdot 10^{-3}$), donde no hay tetraedros diminutos pero sí astillas; los datos son compatibles con que ambos contribuyan, sin permitir separar su efecto. Con malla suave, el iterativo de scikit-fem no es fiable; el directo lo es, pero no escala. Los tetraedros diminutos y las astillas son una limitación de la propia malla TET10, independiente del motor.
- 4 **Espacio jerárquico de NGSolve.** El H1 de orden 2 de NGSolve no es lagrangiano nodal: las funciones de arista son burbujas y solo las de vértice forman partición de la unidad. En la primera versión del adaptador, la reacción sumaba también los GDL de burbuja (0,035 N en lugar de 0,040 N en un bloque TET10) y el plato los imponía, lo que curvaba el techo. La prueba del bloque 29 lo detectó; el adaptador impone el plato y suma la reacción solo en los GDL de vértice (los de burbuja del techo quedan fijos a cero).
- 5 **Newton con plato.** Imponer el incremento del plato solo en el techo llevó a NGSolve, en el bloque al 20 % con St. Venant-Kirchhoff, a otra rama de equilibrio ($-133,4 \text{ N}$ en lugar de $115,2 \text{ N}$). Con el predictor consistente (ecuación 23) los tres motores con Newton propio convergen en tres iteraciones a la solución cerrada.

Dos fallos del mallado TET10 de la propia app aparecieron por el camino y se corrigieron en `fem.preparar_tet10`. El primero: tetgen deja tetraedros «astilla» con sus cuatro esquinas sobre una cara del cubo, y al llevar los nodos a su plano exacto su volumen se hacía cero y la malla se rechazaba (29 elementos en el espinodoide a 32^3 ; la ruta de FEBio habría abortado igual). Se retiran y se declara cuántos. El segundo: los nodos intermedios de las aristas con una esquina ajustada a un plano quedaban hasta $9 \cdot 10^{-4}$ h fuera del punto medio; los motores reciben las aristas rectificadas (ecuación 1).

4.7 Distribución e independencia

Tabla 4. Viabilidad de cada motor dentro del instalador de Windows de spinpy.

Motor	Rueda para Windows en PyPI	Tamaño instalado	Importación	Requisitos en tiempo de ejecución	Licencia
App (spinpy)	sí (numpy, scipy, pyamg)	ya incluida	ya incluida	ninguno	MIT
NGSolve	sí (6.2.2607, cp313)	134 MB	0,48 s	ninguno (MKL opcional, 672 MB, innecesario)	LGPL-2.1
scikit-fem	sí (Python puro)	1,4 MB	0,54 s	ninguno	BSD-3
SfePy	no (solo Linux)	43 MB	0,96 s	compilar desde las fuentes en Windows	BSD-3
FEniCSx	no (solo conda-forge)	3,6 GB (entorno conda)	0,54 s	compilador de C (JIT de FFCx/CFFI) y MPI	LGPL-3.0

La compilación JIT de FEniCSx costó entre 1,5 y 3,4 s la primera vez que se usó cada forma (después queda en caché en disco). En un ejecutable congelado para un usuario sin herramientas de desarrollo, ese paso exige incluir un compilador de C, lo que no es razonable.

Instalador de Windows. El instalador de la V2.0.0, con NGSolve y scikit-fem, se construyó en el servicio de integración continua (Windows, Python 3.13, versiones fijadas en `instalador/requisitos_build.txt`) y su ejecutable pasó la autocomprobación, que resuelve un mismo ensayo lineal con hexaedros y con TET10 con todos los motores presentes y exige que coincidan. Llegar a ello exigió resolver dos problemas de empaquetado que no aparecen al instalar las ruedas en Linux. El primero: PyQt5-Qt5 5.15.2 trae el runtime de C++ `msvcrt140.dll` en su versión 14.26 y netgen, el mallador en que se apoya NGSolve, trae la 14.50 con el mismo nombre; Windows reutiliza la primera que se carga, y como la interfaz carga PyQt5 al arrancar, NGSolve terminaba con una violación de acceso al importarse. La aplicación carga ahora la versión más reciente antes que PyQt5, y el ejecutable lleva una sola copia del runtime, la más reciente disponible al compilar. El segundo: netgen necesita las 47 bibliotecas de OpenCASCADE del paquete `netgen-occt` (unos 49 MB), que se instalan fuera de `site-packages` y que PyInstaller no recoge solo. La carpeta del programa ocupa 860 MB.

5. Discusión

Exactitud. Ninguno de los cuatro motores introdujo un sesgo detectable frente al resolutor de la app ni frente a FEBio. Cuando el sistema se resolvió con un directo, las diferencias fueron del orden del redondeo (10^{-13} a 10^{-11}); cuando se resolvió con un iterativo convergido, del orden de su tolerancia. Esto era lo esperable si cada motor monta la misma matriz, y confirma que el emparejamiento de nodos, la orientación de las caras, la carga consistente y los apoyos se trasladaron bien a cada biblioteca, algo que no se puede dar por supuesto: dos de esos pasos (el espacio jerárquico de NGSolve y las aristas curvas de las mallas de spinpy) habrían producido resultados erróneos sin ningún mensaje de error. Las soluciones cerradas del bloque, lineal y no lineal, y la coincidencia con FEBio en la cavidad sitúan la verificación del código en el nivel que Erdemir et al. (2012) piden a un análisis de elementos finitos en biomecánica: comparación con soluciones analíticas y con otro código verificado.

Fiabilidad frente a exactitud. Las tres excepciones no son errores de exactitud del motor sino de su comportamiento en condiciones difíciles, y las tres se detectaron porque la comparación no se limitó a comprobar que cada resolutor declarase convergencia. El caso de scikit-fem ilustra el riesgo: un residuo relativo de $9 \cdot 10^{-11}$, que cualquier criterio habitual aceptaría, convivió con un error del 0,9 % en desplazamientos y del 7 % en von Mises, porque las mallas suaves de hueso real contienen elementos casi degenerados que elevan el número de condición. En la aplicación, esto justifica usar por omisión un resolutor directo o un preconditionador de subestructuración robusto (BDDC) con TET10, y no un multigrad algebraico genérico. El caso de SfePy recuerda que una tangente inconsistente no altera el resultado cuando el Newton converge, pero sí su convergencia, y en una estructura trabecular basta para no llegar a la carga pedida.

Rendimiento. Con hexaedros, el resolutor de la app (CG con multigrad algebraico en un solo hilo) queda a la par de los iterativos de los demás motores y solo lo superan los directos multihilo: a 661 965 GDL, 128 s frente a 28 s (MUMPS) y 33 s (Cholesky de NGSolve), con una memoria de 2,6 GB frente a 3,5 y 3,4 GB. Con TET10, donde la app no llega, los directos y BDDC resuelven 790 599 GDL en menos de un minuto, mientras que el CG con pyamg tarda un orden de magnitud más. La comparación entre los dos directos de NGSolve fue inesperada: su Cholesky propio fue más rápido que PARDISO en todos los casos (de 1,4 a 1,6 veces) y usó entre 2,3 y 2,8 veces menos memoria. No se investigó la causa; puede depender de la configuración con que NGSolve llama a PARDISO y no debe generalizarse a otros programas. En el no lineal, FEniCSx fue dos veces más rápido que NGSolve una vez compilada su forma; NGSolve redujo a la mitad su tiempo al compilar el árbol de expresiones de la energía, sin compilador externo.

Independencia. El criterio que decide entre los dos motores más rápidos no es numérico. FEniCSx necesita un compilador de C en tiempo de ejecución y solo se distribuye por conda-forge; incluirlo en el instalador de una aplicación de escritorio equivale a volver a depender de herramientas externas, justo lo que se quería evitar. NGSolve se instala desde PyPI en Windows con sus ruedas binarias (134 MB instalado, con netgen), no necesita compilar nada al ejecutarse y su licencia (LGPL-2.1) es compatible con la distribución del ejecutable. Por eso la aplicación usa NGSolve como motor recomendado y conserva los demás como opcionales: si el usuario los tiene instalados en su Python, la aplicación los detecta y los ofrece en «Motor FEM», «Comparar motores» e «Informe FEM (Auto)», lo que permite repetir cualquier ensayo con otro código como verificación cruzada.

6. Limitaciones

- Las medidas se hicieron en un único equipo Linux de 4 núcleos y 15 GB. Los tiempos absolutos cambiarán en otro equipo, y en Windows no se midieron; las proporciones entre motores deberían mantenerse, pero no se comprobó.
- En Windows solo se comprobó que el ejecutable resuelve correctamente un ensayo pequeño con cada motor (autocomprobación, sección 4.7); los tiempos y la memoria no se midieron allí.
- Este estudio compara motores sobre una misma malla y no estudia la convergencia con el refinamiento. Lo medido al respecto está en `comparativa_febio_tet/INFORME.md` (sección 4): en la cavidad esférica, `E_app` de la malla suave quedó a menos del 0,2 % de la referencia analítica en todos los refinamientos, mientras que el pico de von Mises no convergió (osciló entre $-1,1\%$ y $+12,1\%$ con los parámetros de suavizado por omisión). Los resultados de rigidez son, por tanto, más robustos a la discretización que los de tensión máxima.
- Se evaluó un material elástico isotrópico y dos hiperelásticos; no se evaluaron plasticidad, contacto ni daño, que ninguno de los flujos de la app usa.
- Las cotas de homogeneización KUBC/SUBC con TET10 que daba la ruta de FEBio no se reimplementaron; la homogeneización periódica con hexaedros de la app no cambia.
- Con control por fuerza, los motores aplican una tracción muerta y FEBio una presión seguidora; a las cargas de los protocolos la diferencia es del orden de la rotación de las caras del techo, pero no se cuantificó.
- La tolerancia de los resolutores iterativos (10^{-10}) se fijó antes de medir; con malla suave no garantiza por sí sola la exactitud (sección 4.6).

7. Reproducir

```

pip install -r requirements.txt                # app + NGSolve + scikit-fem
pip install sfepy                             # opcional (Linux)
# FEniCSx, opcional: conda create -c conda-forge -n fx fenics-dolfinx
cd comparativa_motores
FENICSX_PYTHON=/ruta/al/python/de/fx python correr.py exactos nl cavidad espinodoide_hex espinodoide_tet
SPINPY_SIN_PARDISO=1 python correr.py espinodoide_hex --n-hex 48 64 80 --motores ngsolve --solvers directo --gdl-directo-hex 1e7
python figuras.py                             # tablas.md y figs/
python -m pytest ../tests/test_29_motores.py   # versión pequeña, ~20 s

```

La campaña completa tardó unas tres horas en el equipo descrito. Los resultados quedan en `resultados/*.jsonl`, una línea por corrida.

8. Referencias

Verificadas en PubMed (metadatos y DOI):

- 1 Erdemir A, Guess TM, Halloran J, Tadepalli SC, Morrison TM. Considerations for reporting finite element analysis studies in biomechanics. *J Biomech.* 2012;45(4):625-33. doi:10.1016/j.jbiomech.2011.11.038
- 2 Maas SA, Ellis BJ, Ateshian GA, Weiss JA. FEBio: finite elements for biomechanics. *J Biomech Eng.* 2012;134(1):011005. doi:10.1115/1.4005694
- 3 Pistoia W, van Rietbergen B, Lochmüller EM, Lill CA, Eckstein F, Rügsegger P. Estimation of distal radius failure load with micro-finite element analysis models based on three-dimensional peripheral quantitative computed tomography images. *Bone.* 2002;30(6):842-8. doi:10.1016/s8756-3282(02)00736-6

- 4 van Rietbergen B, Weinans H, Huiskes R, Odgaard A. A new method to determine trabecular bone elastic properties and loading using micromechanical finite-element models. *J Biomech.* 1995;28(1):69-81. doi:10.1016/0021-9290(95)80008-5

Verificada con su texto completo en arXiv (1810.00674, preprint enviado a *Advances in Computational Mathematics*); el volumen, las páginas y el DOI de la versión publicada no pudieron comprobarse en línea:

- 1 Cimrman R, Lukeš V, Rohan E. Multiscale finite element calculations in Python using SfePy. *Adv Comput Math.* 2019;45:1897-921. doi:10.1007/s10444-019-09666-0

Publicaciones de los motores y algoritmos usados. Título, autores, revista, año y DOI se cotejaron en el índice de SciSpace (6 a 10); volumen y páginas proceden de la cita original y no pudieron cotejarse en línea, porque Crossref y doi.org no fueron accesibles desde el entorno de trabajo:

- 1 Baratta IA, Dean JP, Dokken JS, Habera M, Hale JS, Richardson CN, et al. DOLFINx: the next generation FEniCS problem solving environment. Zenodo; 2023. doi:10.5281/zenodo.10447666
- 2 Gustafsson T, McBain GD. scikit-fem: a Python package for finite element assembly. *J Open Source Softw.* 2020;5(52):2369. doi:10.21105/joss.02369
- 3 Van der Vort P, Mandel J, Brezina M. Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems. *Computing.* 1996;56(3):179-96. doi:10.1007/BF02238511
- 4 Dohrmann CR. A preconditioner for substructuring based on constrained energy minimization. *SIAM J Sci Comput.* 2003;25(1):246-58. doi:10.1137/S1064827502412887
- 5 Amestoy PR, Duff IS, L'Excellent JY, Koster J. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM J Matrix Anal Appl.* 2001;23(1):15-41. doi:10.1137/S0895479899358194

No encontradas en los índices consultados (PubMed, SciSpace); deben cotejarse antes de citarlas en un manuscrito:

- 1 Schöberl J. C++11 implementation of finite elements in NGSolve. ASC Report 30/2014. Vienna: Institute for Analysis and Scientific Computing, Vienna University of Technology; 2014.
- 2 Bonet J, Wood RD. *Nonlinear continuum mechanics for finite element analysis*. 2nd ed. Cambridge: Cambridge University Press; 2008.

9. Lista de chequeo de la guía de reporte

Diseño. Estudio computacional de verificación de código y comparación de desempeño entre programas de elementos finitos, sin participantes humanos ni animales. Las guías de reporte de la red EQUATOR para estudios clínicos u observacionales (CONSORT, STROBE, PRISMA, STARD, TRIPOD) no se aplican. La guía pertinente es la de consideraciones para el reporte de análisis de elementos finitos en biomecánica (Erdemir et al., 2012), cuyas categorías (identificación del modelo, estructura del modelo, estructura de la simulación, verificación, validación y disponibilidad) se revisan a continuación.

Categoría e ítem (Erdemir et al., 2012)	Estado	Dónde / modificación necesaria
Identificación del modelo		
Propósito del modelo y de la simulación	Cumple	Sección 1
Identificador y versión del modelo y de los programas	Cumple	Sección 1 (tabla de motores y versiones); spinpy V2.0.0
Estructura del modelo		
Geometría y su origen (imagen, generación)	Cumple	Sección 2.2 (espinodoide con parámetros y semilla; bloque; cavidad)
Discretización: tipo de elemento, orden, número de elementos y GDL	Cumple	Secciones 2.2, 3.2 y Tablas 1 y 2

Categoría e ítem (Erdemir et al., 2012)	Estado	Dónde / modificación necesaria
Calidad de malla (ángulos, relación de aspecto)	Cumple	Sección 4.4 y Tabla 3 (relación radio-arista, diedro mínimo, elementos diminutos); datos en resultados/calidad_tet10.json
Propiedades de los materiales y ley constitutiva	Cumple	Secciones 2.2 y 3.5 (E, v, SVK, neo-Hookeano)
Condiciones de contorno y cargas	Cumple	Sección 3.1
Interacciones y contacto	No aplica	No hay contacto; el plato se modela como desplazamiento impuesto
Estructura de la simulación		
Tipo de análisis (estático, lineal o no lineal)	Cumple	Secciones 3.1 y 3.5
Resolvedores y tolerancias	Cumple	Secciones 3.3 y 3.5
Plataforma de cálculo (hardware, hilos)	Cumple	Sección 2.3
Verificación		
Verificación del código frente a soluciones analíticas	Cumple	Secciones 4.1 y 3.6; bloque 29 de la suite
Verificación frente a otro código verificado	Cumple	Sección 4.2 (FEBio) y concordancia entre cinco motores
Verificación del cálculo: convergencia de malla	Cumple parcialmente	Este estudio compara motores sobre la misma malla; la sección 6 resume la convergencia medida en comparativa_febio_tet/INFORME.md (E_app estable a < 0,2 %; el pico de tensión no converge). Para un manuscrito, correr resistencia.estudio_convergencia sobre el espécimen analizado
Convergencia del resolutor (residuo, Newton)	Cumple	Secciones 3.3, 4.5 y 4.6; residuo e iteraciones por corrida en tablas.md
Validación		
Comparación con datos experimentales	No aplica	El objetivo es verificar la implementación de los motores, no validar el modelo de hueso; la validación frente a resultados publicados está en Test/ y docs/validacion_literatura/
Disponibilidad		
Modelo, datos de entrada y resultados accesibles	Cumple	comparativa_motores/ (código, casos, resultados JSONL, figuras) en el repositorio
Programas y licencias	Cumple	Sección 4.7 (Tabla 4)
Otros		
Incertidumbre y sensibilidad de los resultados	Cumple parcialmente	Se informa la dependencia de la tolerancia del resolutor y el efecto de la malla en la cavidad; no se estudió la sensibilidad a los hilos ni la variabilidad de tiempo entre repeticiones. Repetir cada medida de tiempo tres veces e informar la mediana y el rango
Limitaciones	Cumple	Sección 6